

How Modern Is LLM-Generated C++?

An Empirical Evaluation of Modern C++ Coding Practices

Swetha Govindaiah | Software Engineer | Ph.D. in Computer Science | swethagovindh@gmail.com

Empirical insights for
responsible and effective
use of AI-generated C++

1. Motivation & Research Questions

Large Language Models (LLMs) are increasingly used to generate C++ code. While the generated code is often syntactically correct and functional, it is unclear how closely it follows modern C++ design principles and engineering practices.

This study asks:

- Does AI-generated code follow modern C++ practices?
- Which modern language features are consistently adopted?
- Which best practices are frequently overlooked?
- Where should developers exercise additional review before integrating generated code into production systems?

2. Evaluation Framework (11 Families)

Based on Modern C++ standards, C++ Core Guidelines, secure-coding guidance, and domain references.

- Memory Safety** – RAII, smart pointers, lifetime management, ownership
- Type Safety** – strong typing, enum class, concepts, safe casting
- Bounds & Data Safety** – safe containers, `std::span`, buffer safety
- Compile-Time Enforcement** – `constexpr`, `constexpr`, `static_assert`
- Modern Language Feature Adoption** – C++17/20/23 features, appropriate selection
- Standard Library & Generic Programming** – STL containers, algorithms, ranges
- Modern Programming Style** – lambdas, structured bindings, `constexpr`
- Concurrency & Parallelism** – threading, atomics, synchronization where applicable
- Error Handling & Robustness** – exceptions, `std::optional`, `std::expected`
- Platform & Systems Awareness** – alignment, low-level memory, portability
- Code Quality & Maintainability** – readability, modular design, compile success

3. LLM Models Evaluated

Model	Provider	Access
GPT-5.6 Sol	OpenAI	API
Claude Opus 5	Anthropic	API
Gemini 3.1 Pro Preview	Google	API

Note: Three major contemporary LLM families are evaluated using the same prompts and tasks. The goal is to analyze practices, not to rank models.

4. Prompt Types

ID	Prompt Type	Focus
P0	Baseline	General task description; no explicit Modern C++ emphasis
P1	Modern C++	Use Modern C++ practices and language/library features where applicable
P2	Safety / Robustness	Prioritize safety, security, robustness, and correctness
P3	Performance	Prioritize performance, low latency, and efficient resource usage

5. Programming Scenarios (10 Tasks)

ID	Scenario / Task	Domain	Key Concepts
S01	Limit-Order Book Matching	Fintech / Electronic Trading	matching, low latency, data structures
S02	Rendering Loop	Game Development	rendering, resource management
S03	Sensor Fusion and Spatial Mapping	Autonomous Systems	fusion, data flow, safety
S04	Telemetry and Guidance Processing	Aerospace	telemetry, correctness, robustness
S05	Inter-Process Data Exchange	Operating Systems / Middleware	IPC, interfaces, reliability
S06	Memory-Mapped Device Interface	Embedded Systems	low-level I/O, portability
S07	Concurrent Multi-Threaded Queue	Concurrent Systems	threads, synchronization, ownership
S08	Large-Scale Data Storage and Aggregation	Database / Search Systems	storage, aggregation, scalability
S09	Dynamic Object Lifecycle Management	Browser-like Systems	object lifecycle, ownership
S10	Large-Scale C++ Component and Dependency Design	Enterprise Infrastructure	architecture, modularity, interfaces

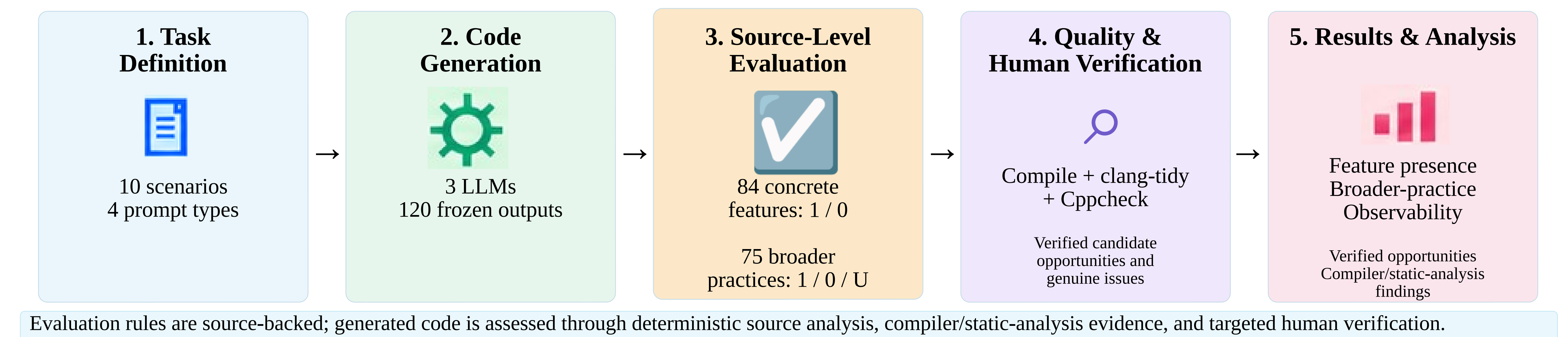
7. Evaluation Criteria

Concrete Feature Presence (84 features)	Broader Practices (75 practices)	Code Quality & Correctness
- Binary detection from source code - Present = 1 Absent = 0 - Descriptive measure, not a quality score	- Practice-level evaluation - Present = 1 Absent = 0 - Undetermined = U when evidence is insufficient	- Compiler: Clang 22.1.8, C++23, -Wall -Wextra -Wpedantic - Static analysis: clang-tidy 22.1.8, Cppcheck 2.21.0 - Targeted human verification

10. Reproducibility

GitHub repository Clone for code or data	10 benchmark scenarios × 4 prompts 120 frozen V3 LLM responses 202-row source-backed evaluation inventory 84 concrete-feature + 75 broader-practice evaluation data - Analysis scripts + static-analysis configuration	- Frozen human-verified opportunity/defect findings - Result aggregation + figure-generation scripts - README + one-command reproduction pipeline	
---	--	---	---

6. Evaluation Pipeline



9. Key Takeaways

- Targeted Modern C++ and Safety/Robustness prompts increased concrete-feature presence relative to Baseline.
- Feature presence varies by task, model, and prompt type.
- Broader practices are less directly observable: 36.1% of 9,000 evaluations remained Undetermined (U).
- Among verified opportunities, bounds-aware interfaces were the clearest recurring overlooked practice, especially raw pointer + size/count APIs.
- Feature presence is not a quality score; compiler/static analysis identified 10 verified defect instances plus 2 portability issues.
- Context-dependent findings require **human verification**, particularly for API/error semantics, architecture, and low-level boundaries.

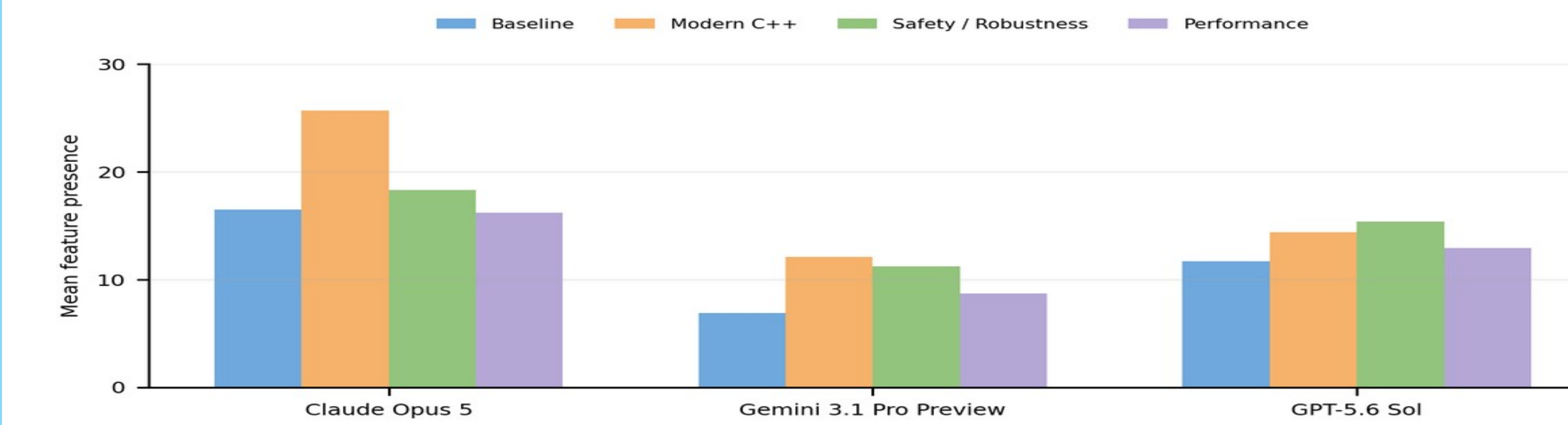
This study helps developers understand Modern C++ feature presence, broader-practice observability, and where human review is needed in LLM-generated C++.

11. Sources & Tools

Source basis	Analysis tools
- C++ Core Guidelines — primary best-practice guidance - ISO/IEC 14882:2024 – normative C++ standard through C++23 - cppreference — feature/version verification - SEI CERT C++ — supplemental security/correctness guidance - MISRA C++:2023 — supplemental safety-critical guidance - Domain/context refs — Chromium, RocksDB, CMSIS, Boost, Google/LLVM, platform docs	- Custom Python analysis scripts - Clang 22.1.8 — C++23, -Wall -Wextra -Wpedantic - clang-tidy 22.1.8 — analyzer, bugprone, performance, modernize, readability - Cppcheck 2.21.0 — warning, style, performance, portability - Targeted human verification

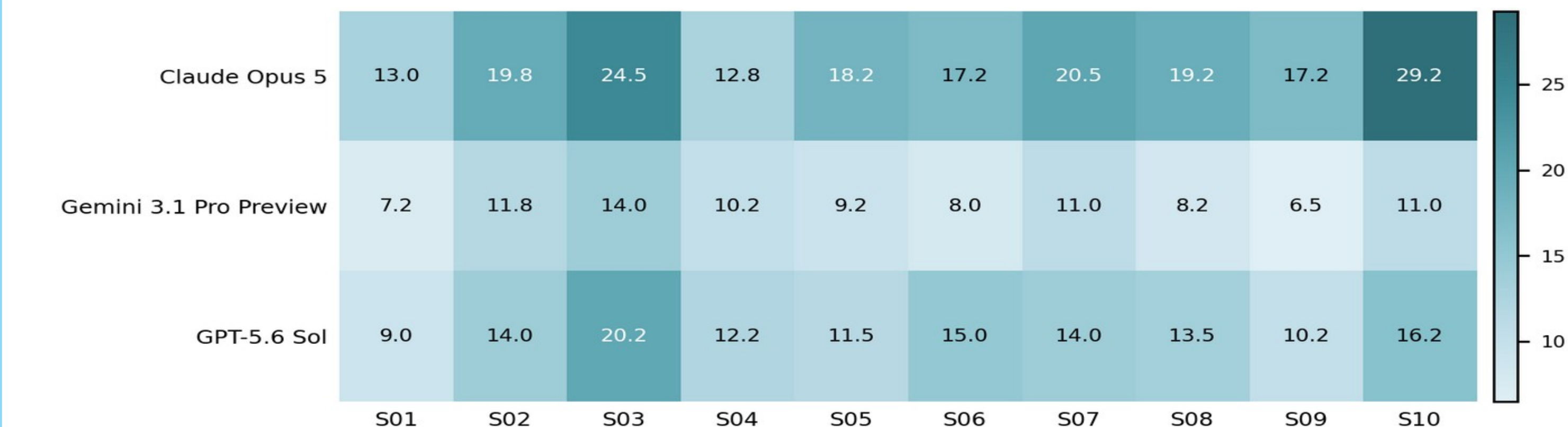
8. Results

Figure 1. Concrete Feature Presence by Model and Prompt



Modern C++ and Safety/Robustness prompts result in higher concrete feature presence across models.

Figure 2. Scenario-Level Concrete Feature Presence (averaged across the four prompt types)



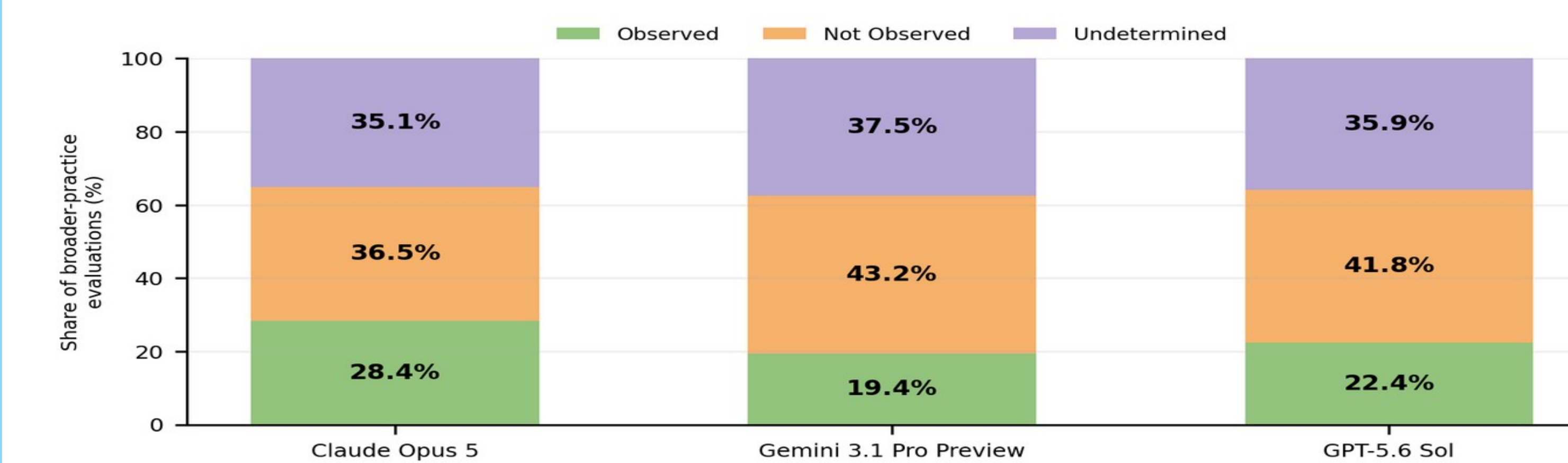
Concrete-feature presence varies substantially by programming scenario.

Figure 3. Scenario-Level Concrete Feature Presence by Prompt



Modern C++ prompts show higher feature presence in many scenarios, with variation across tasks.

Figure 4. Broader-Practice Observability (by Model)



Overall Undetermined: 36.1% (3,253 of 9,000 evaluations). Not Observed does not imply a flaw or overlooked opportunity; the practice may not be required by the task.

Overlooked Practices (Key Finding)

Observed overlooked practices: 11 high-confidence, manually verified opportunities, primarily in Bounds & Data Safety. Additional context-dependent opportunities may exist beyond this verified set.